

Informfully Research Platform - An Open Source Project for Conducting Empirical Research with Recommender Systems

LUCIEN HEITZ, Department of Informatics, University of Zurich, Zurich, Switzerland and Digital Society Initiative, University of Zurich, Zürich, Switzerland

JULIAN A. CROCI, Department of Informatics, University of Zurich, Zurich, Switzerland

MADHAV SACHDEVA, Department of Informatics, University of Zurich, Zurich, Switzerland

ABRAHAM BERNSTEIN, Department of Informatics, University of Zurich, Zurich, Switzerland

This resource paper presents Informfully, an open source software project for content distribution to support user studies. It is a free resource for the research community to help conduct online studies investigating the impact of item recommendations on users' consumption behavior. Informfully allows for the push of text, image, audio, and video content to users and automatically generates a detailed log of their consumption history. It features three core components: 1) a front end for displaying and interacting with recommended items, 2) a back end for researchers to create and maintain user experiments, and 3) a simple JSON-based exchange format for ranked item recommendations to interface with third-party frameworks. We provide a system overview and outline the platform's three core components. A sample workflow is presented for conducting field studies that incorporate multiple user groups, personalize recommendations, and measure the impact of algorithms on user engagement.

CCS Concepts: • **Information systems** → **Recommender systems**; **Presentation of retrieval results**; **Users and interactive retrieval**; • **Human-centered computing** → **Open source software**.

Additional Key Words and Phrases: news recommender system, content distribution platform, open-source software

1 Introduction

Informfully offers a solution for setting up user studies, creating experiment groups, distributing media content, prompting questionnaires using an in-app survey tool, and automatically recording all interactions (i.e., access time, access duration, item ratings/feedback, and other engagement metrics). The goal of this resource is to offer the research community an easy-to-use and customizable solution for conducting online experiments and to promote empirical studies by lowering the barrier to entry. Running such real-world experiments is a crucial part of research, as they present the ultimate test for evaluating the impact of recommender systems (RS) and an essential element to counter the “leaderboard-chasing culture” [29] of offline experiments.

Research in the domain of RS has shown that online evaluations are rare [30], can fall short of predicting actual user interactions with the systems [6], or feature too few participants to provide a meaningful assessment [5]. Offline evaluations can also fall short of predicting real-world user interactions, as not only do users' opinions and preferences matter, but also how the items are shown to users [37]. The way items are presented and placed on screen can significantly impact user selection [3, 4], and addressing this remains an open problem for

Authors' Contact Information: Lucien Heitz, Department of Informatics, University of Zurich, Zurich, Switzerland and Digital Society Initiative, University of Zurich, Zürich, Switzerland; e-mail: lucienheitz@msn.com; Julian A. Croci, Department of Informatics, University of Zurich, Zurich, Switzerland; e-mail: croci@ifi.uzh.ch; Madhav Sachdeva, Department of Informatics, University of Zurich, Zurich, Switzerland; e-mail: sachdeva@ifi.uzh.ch; Abraham Bernstein, Department of Informatics, University of Zurich, Zurich, Switzerland; e-mail: bernstein@ifi.uzh.ch.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2770-6699/2026/6-ART

<https://doi.org/10.1145/3820883>

automated/offline testing [9]. Therefore, a thorough evaluation of an RS should include online studies to address interface considerations and better estimate the real-world impact on users.

Studying the real-world implications of RSs—especially in sensitive areas like news recommendations, which affect opinion-making and inform political decision-making [7, 25, 26, 44]—is absolutely critical for a reliable assessment of the risks associated with deploying RS and for analyzing users’ needs (cf. [14]). However, running real-world experiments is complex—it requires coordinating with multiple actors, including content providers and users—and entails development costs that require a sizable investment in software scaffolding. As a secondary effect, the cost and complexity of such experiments, combined with their rarity, significantly hinder their reproducibility.

To reduce this complexity and the costs of deploying RSs, we built Informfully [18], a content aggregation and distribution platform that enables researchers to set up and run empirical studies to track user behavior and item interactions with text, images, audio, and video in real time.¹ Informfully enables reproducible experiments by sharing the experiment configuration files to rerun studies and compare results across experiments. This allows researchers to export the settings of their user studies and to re-launch them with new participants. It introduces a content-agnostic format for exchanging item recommendations, enabling integration with external RS frameworks and supporting multiple media types. Furthermore, the tool features a web-based control panel that allows users to create and configure their experiments. Account creation, group assignment, item curation, and questionnaire creation are out-of-the-box features. In addition, the platform offers a native app for users to consume, rate, and interact with recommended items. Taken together, this enables researchers to verify their RS pipeline end-to-end, thereby improving the external validity of assessments of how users interact with the system in a real-world setting.

This paper discusses related work, introduces Informfully, and evaluates it by showcasing results from previous user studies. By default, Informfully is content agnostic. For the purpose of this paper, however, we pick the domain of news recommendations as an example use case.

2 Related Work

Informfully is not the first tool to assist researchers in conducting field studies. The existing solutions, however, have disadvantages that negatively impact their reusability. We limit the analysis of related work to solutions in the domain of RS that offer a dedicated tool with the capabilities of 1) capturing and storing user interactions (i.e., simple interactions, such as item clicks) with recommended items in experimental user studies, as well as 2) means for presenting the output of RS to users. Our literature review consists of two parts. First, Table 1 provides an overview of existing tools for conducting empirical research and for what purpose they have been used in the past. Second, Table 2 looks at key usability and accessibility features of tools for conducting user studies.

Tool Overview. Table 1 presents an overview of tools from the literature that were used to conduct user studies with an interface to present the recommended items. We list the setup and purpose for which they have been used by examining the supported platform dimensions, domain, target audience, customizability, item modalities, and the codebase licensing.

¹Official Informfully website: <https://informfully.ch>

GitHub code repository: <https://github.com/informfully/platform>

Online platform documentation: <http://informfully.ch/docs>

Table 1. Overview of tools found in the literature that were used for conducting empirical studies with RS that present recommended items and record user interactions.

Tool	Platform	Domain	Design Intent	Customizability	Modalities	Codebase
PNS [39]	web	news	academia	monolithic	t, i	×
MovieExplain [48]	web	movies	academia	monolithic	t, i	×
PEN [12]	web	news	academia	modularized	t, i	×
NZZ Compendium [31]	mobile	news	commercial	monolithic	t, i	×
3bij3 [33]	web	news	academia	modularized	t	✓
Music RS [35]	web	music	academia	monolithic	t, i, a	×
Movies RS [49]	web	movies	academia	monolithic	t, i	×
Blendle [36]	web	news	commercial	monolithic	t, i	×
EasyStudy [10]	web	general	academia	monolithic	t, i	✓
Informfully	mobile, web	general	academia	modularized	t, i, a, v	✓

Platform: A broad categorization is made based on the platforms supported. We differentiate between web-based solutions (i.e., websites) and native mobile apps.

Domain: This dimension captures the intended application domain. We distinguish between tools for news, movies, music, and general-purpose recommendations.

Audience: This dimension lists the platform’s original purpose. We distinguish between commercial products and solutions targeting academia/research.

Customizability: We differentiate between monolithic and modularized software [46]. Monolithic tools combine the user interface and RS in a single **static** pipeline that cannot be altered. Modularized tools are more flexible and allow for the addition of custom components.

Modalities: The modalities column lists what types of items the tool can recommend. The types included here are text (t), image (i), audio (a), and video (v).

Codebase: We mark an entry with ✓ if the tool’s codebase is open-source. If a tool is proprietary and/or the codebase is no longer accessible, it is marked with ×.

We observe that most tools focus on browser deployment. Only two of the web-based tools explicitly mention and consider mobile interface design [33, 36]. Against the backdrop of the increasing popularity of mobile apps and the in-depth tracking they enable [28], Informfully was designed with a reactive interface to fit both platforms. Looking at the target domains of the tools, we see that existing research features custom-built solutions for a single item. Only one of the existing works has explored general-purpose recommendations [10]. Most offerings target academia and researchers, covering a wider range of user cases. The two commercial solutions [31, 36] both exclusively focus on news and are tied to a specific outlet. In terms of customizability, we assess the extent to which a platform enables researchers to modify components, such as the recommender logic or content sources. Platforms with low customizability are typically monolithic, representing the majority in our comparisons. We see that all monolithic solutions are either tied to a specific recommender algorithm [12], data provider [31, 36], item type/modality [10, 35, 49], or a combination thereof [39]. This may negatively impact the reusability and comparability of research results across experiments. Highly customizable systems typically feature a more modular approach, which defines component interfaces for communication and allows components to be replaced with minimal coding effort [46]. These modular approaches, however, are a minority [12, 33]. As far as item modalities go, all solution support text. All but one tool [33] can display images. And only one existing tool supports multimedia items, but it is limited to audio files [49]. Finally, a key element of reproducible research is the availability of the source code. The majority of tools are not publicly available [35, 39, 48, 49] and/or are proprietary [12, 31, 36].

Feature Overview. Usability and accessibility are important aspects when designing software that is used for conducting user studies [40]. Making a tool easy to use and visualizing recommendations can have a significant impact on engagement and retention [4]. The literature on usability and interface design points out the following aspects that should be considered: ratings that can be updated and modified [1], responsive design (scaling of the interface) [45], tutorials for guiding the user through the platform’s functionalities [38], and configuration websites for setting up and configuring the experiments [10]. Furthermore, by looking at commercial products that recommend news, music, or video, we identified the following additional dimensions: color modes (e.g., light and dark mode), font size (adjust the size of text and interface elements), localization (provide different translations for labels and buttons), and surveys for users to provide feedback. Table 2 shows an overview of these features together with a list of platforms that include them.

Table 2. Overview of usability and accessibility features, together with a list of tools that include them.

Feature	Tools
Color Modes	Informfully
Font Size	Informfully
Localization	Informfully
Ratings	PNS, Informfully
Responsive Design	NZZ Compendium, Informfully
Surveys	Informfully
Tutorial	EasyStudy, Informfully
Website	EasyStudy, Informfully

Positioning Informfully. Apart from isolated features being incorporated in some tools, usability and accessibility remain largely unaddressed by existing platforms and experimental setups. However, these are all key features that allow experiment designers to increase the external validity of their results. This increase in external validity was the primary motivation for us to ensure that Informfully supports everything researchers need to conduct experiments that yield reliable results. Compared to alternative offerings, Informfully is a mobile-first front end that provides researchers with the tools to launch and conduct user-focused online RS studies. It allows for standardized item presentation and fully automated collection of user interactions (e.g., recording clicks, reading time, etc.). However, it is not a content curation system.² It requires a dedicated back end for collecting items and calculating recommendation scores and is designed to easily accommodate input from the most frequently used reproducibility frameworks.³

3 Platform Overview

The Informfully Research Platform offers two main interfaces that support conducting and managing user studies: 1) a *smartphone app* for users to interact with recommendations and 2) an *administration website* for experimenters to create and maintain user studies. Section 3.1 highlights the features and functionalities of the app interface that are accessible to participants of these studies. Section 3.2 focuses on the administration website for setting up experiments. Finally, Section 3.3 brings the two parts together by providing a sample workflow for conducting a user study.

²See *Informfully Recommenders* for a complementary open source solution that includes content scrapers and recommender algorithms [20, 32].

³Official list of the ACM RecSys evaluation frameworks: <https://github.com/ACMRecSys/recsys-evaluation-frameworks>

3.1 Smartphone App

Informfully is powered by React Native, which enables the building of platform-independent apps in JavaScript. It is available for both Android (from version 5.1) and iOS (from version 13). Extra care was taken to ensure parity between the two versions. Distribution and installation outside the storefront environment is possible using a separate Android Package Kit (APK) file.⁴

Figure 1 presents a collection of screenshots from the mobile app, showcasing the look and feel of Informfully in the text/image setting for news recommendations. The mobile app provides five main screens to interact with content: A) the item list on the home screen,⁵ B) the detail view containing the previously selected item on the home screen, C) item ratings at the bottom of the detail view, D) lists for bookmarks and favorites, and E) the built-in survey tool for prompting questions.⁶ Users can access items previewed on the home screen even when the app is offline. Items previewed in the app are pre-loaded and locally stored, enabling offline use. All offline interactions are automatically cached and sent to the database once the mobile device reconnects to the internet.

In the detail view, multiple questions, followed by thumbs-up and thumbs-down buttons, are displayed below each item, allowing users to rate the item recommendation (see Figure 1C). These questions enable a more precise assessment of user interaction. The text and the number of questions can be customized for each experiment. User ratings can be followed by a pop-up window that lets users add additional information to their ratings. The thumbs-up/thumbs-down ratings and the pop-up windows can be configured or disabled when setting up an experiment.⁷

Users can add items to either their archive or their reading list (see Figure 1D). Each action is recorded separately, and it is up to the experiment coordinators to decide how they want to instruct participants to use the favorite and bookmark lists. During user studies, however, participants can be instructed to use them differently, enabling researchers to collect richer user feedback. To enable experimentation with participants who are less familiar with apps, we integrated a customizable user tutorial to guide them in using the app (accessible via the settings menu). The tutorial demonstrates the various functionalities of each interface element shown in Figure 1A-D, explains how to manage favorite/bookmark items, and outlines the navigation controls.

3.2 Administration Website

The administration website enables researchers to 1) set up experiments, 2) create user accounts and group assignments, 3) define feedback surveys, and 4) browse items presented to their experimental groups. The upcoming section provides a more detailed description of all these steps and processes.⁸

Creating Experiments. Administrators can create multiple experiments and run them simultaneously with a completely separate set of participants (cf. Figure 2A). The initial experiment status is **Design**. At this stage, the administrator can add users, modify surveys, and experiment with the setup. Once the status is switched to **Launched** by clicking the respective button, the settings are locked to preserve the experimental integrity.

User Management. New accounts for experiments can be added in the user management section (cf. Figure 2B). Researchers can specify the number of users and the user group for account generation. Group membership is a

⁴This APK file also allows the app to be installed on Windows PCs (e.g., for conducting experiments in a lab setting).

⁵For faster and more efficient content delivery, media items are loaded on demand as the user scrolls through the home screen. A notification overlay on the top of the screen will inform the user if new items are available on the home screen.

⁶Figure 1 shows a news item with an image and text. When recommending audio or video items, the image on the home screen is a thumbnail that will be replaced by a media player in the detail view.

⁷Ratings can be customized. Using the editor on the website, users are able to define their own feedback questions: <https://informfully.ch/docs/experiment.html>

⁸The administration website uses a MongoDB database and MeteorJS server for the backend system and React for the front end (i.e., website). Since Meteor uses a WebSocket, any change to the database is immediately reflected on the client and vice versa, enabling real-time tracking.

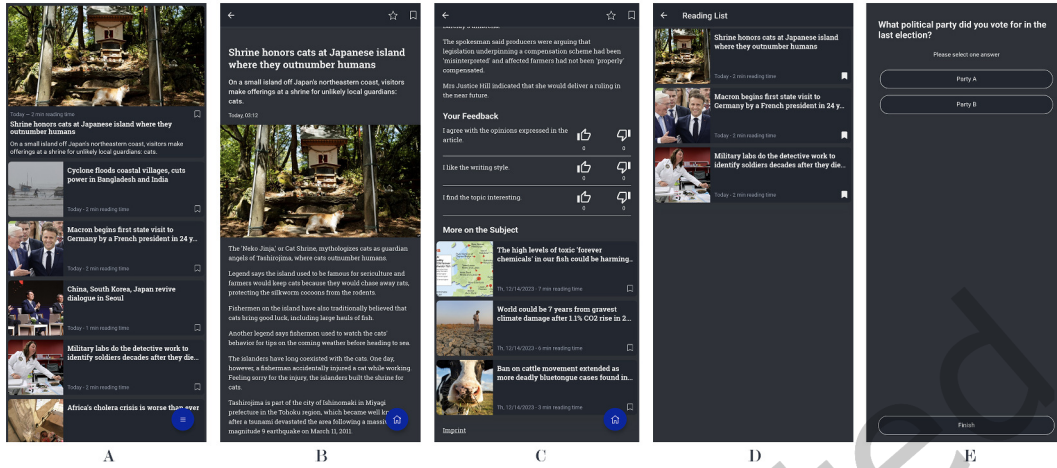


Fig. 1. Informally screenshots from the app interface with the homepage (A), detailed item view (B), item rating and feedback (C), bookmark/favorite list (D), and user survey (E). Screenshots show the app in dark mode on Android.

text string that can be edited. New accounts can be added on demand (e.g., when additional recruitment waves are needed). The back end generates them with randomly generated usernames and passwords. These credentials are available only once during account creation and can be exported to send to participants.

Survey Tool. The app features a built-in survey tool that allows participants to provide information through questionnaires. Using the survey tab (cf. Figure 2C), administrators can create questionnaires at the experiment level. Questionnaires are always mandatory, meaning users must complete them before they can continue browsing and reading news in the app. An administrator can add as many questions and answers as needed to such a survey. The app supports single-choice, multiple-choice, and free-text questions. The survey tool also allows researchers to define follow-up questions to display to users after they have provided their answers (e.g., to request more details on why a specific answer was chosen).

Content Preview. Using the content preview screen (cf. Figure 2D), administrators can get a better picture of how the scraped content will be presented to users. This interface provides an overview of all item titles, leads, and images. Upon selecting a news item, administrators can read the full text and other details, including publication dates and sources. However, this is only for quality control. Assigning recommendations to users or collecting/inserting items is done externally and then forwarded through the appropriate interfaces. We refer to Section 4.4 for a more in-depth description of how to collect and properly format items for feeding into the system. Please see Section 4.5 for details on connecting external RSs to Informfully to receive item recommendations for individual users.⁹

3.3 Experiment Overview

Informfully allows for custom workflows when setting up user experiments. Figure 3 provides an example of how such a workflow could look. The experiment centers on recruiting participants, exposing them to various item recommendations, recording their interactions, and concluding with a debriefing via an exit survey. The upcoming sections will go into detail on each phase.

⁹Please see the online documentation for a complete description of the setup process: <https://informfully.ch/docs/recommenders.html>

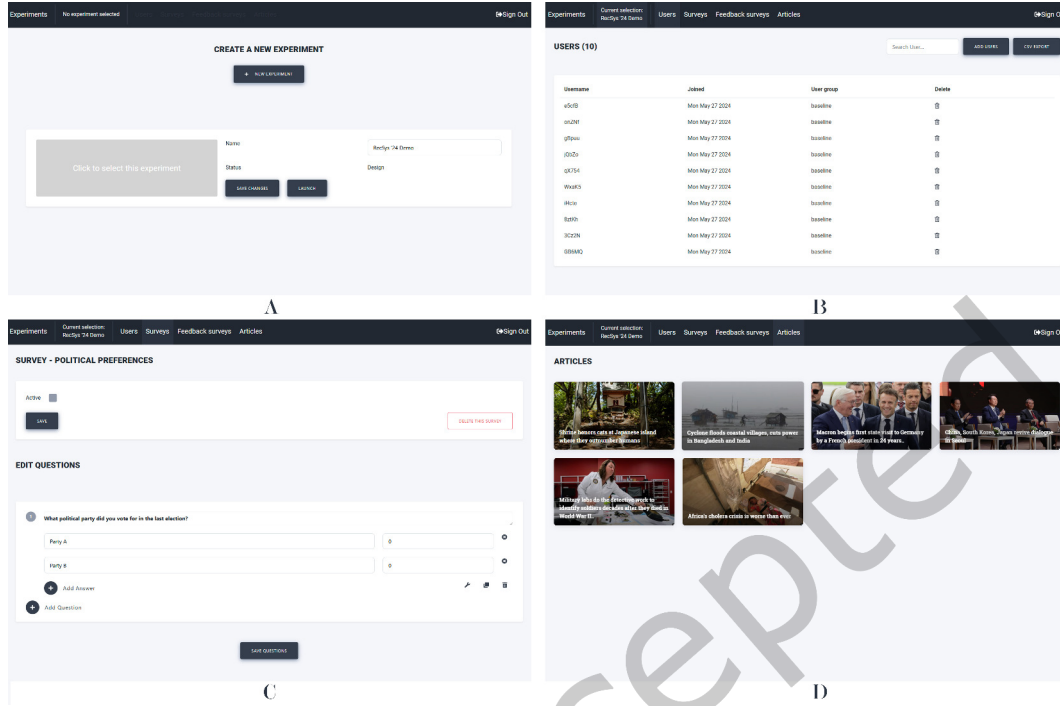


Fig. 2. Informfully screenshots from the website interface with the pages for experiment creation (A), user management (B), survey tool (C), and content preview (D). The administrator interface is built inside a reactive website. The entire interface can be scaled to fit mobile clients.

Phase 1: Setup. Using the administrator interface, a new experiment is created, and new user accounts are registered. The credentials for these new accounts can be exported to a CSV file and distributed to participants, along with a download link to the app. This allows Informfully to be used alongside crowdsourcing platforms. Alternative options for recruiting participants in past user experiments included third-party market research companies creating a panel, which received a CSV file with credentials to be distributed (exported via the web interface), and bulk mailing to potential subjects. The app onboarding can be complemented with entry surveys (e.g., to elicit user preferences for profile creation).

Phase 2: Experiment. The core part of the experiment is the recommendation-feedback cycle. The back end forwards recommended items to users and automatically records their interactions. For this to happen, the system requires scrapers to provide items and one or more external RS frameworks to calculate recommendations. The scraping and recommendation processes happen concurrently to participants using the app (see Background Processes at the bottom of Figure 3). During this phase, both the scraper and the recommender run independently of any other background processes in the experiment. They can access the item collection and recommendation list at any point in time; changes to existing items or recommendations are automatically sent to users.

Phase 3: Debriefing. The final phase of an experiment typically includes an exit survey, followed by the closing/de-registration of user accounts. Alternatively, users can be moved into a subject pool for future experiments. When added to a pool, accounts are either temporarily frozen (participants are unable to access app

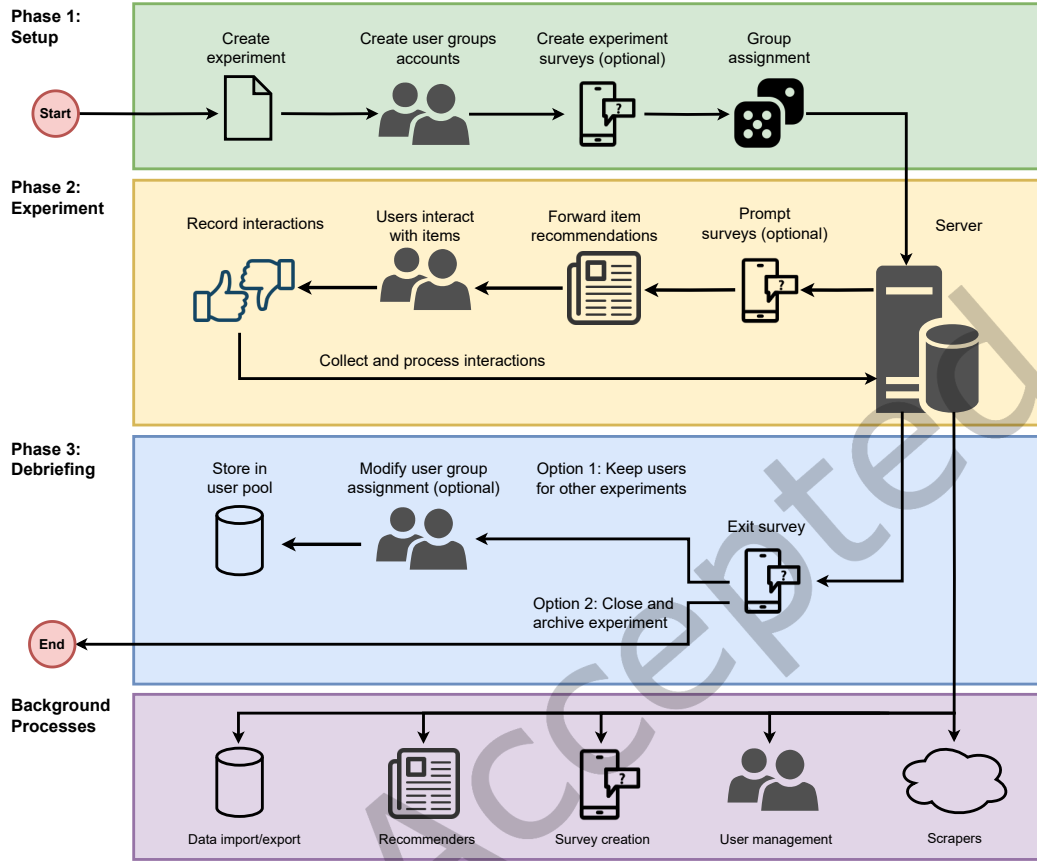


Fig. 3. Overview of the experimental phases of a user study conducted with Informfully. The phases include: 1) study and experiment setup, 2) experimentation with user recommendations, and 3) debriefing and closing of the experiment. In parallel with these three phases, background processes maintain the experiment.

content) or assigned to experiment-specific default items. Experiment termination and de-registration can be triggered at any point in time. These actions, however, currently require manual intervention by the researchers.

Background Processes. The background processes for managing the experiment include scraping routines, recommendation list exchanges, user management, survey creation and evaluation, and data import and export. These processes can be executed at the discretion of researchers at any point in time during all phases.

Supporting Reproducibility. Supporting reproducibility of scientific research has been a guiding principle of Informfully. It does so through code availability, standardized item presentation, and standardized interaction recordings. In terms of code availability, Informfully provides an open source codebase with a complete commit history. When conducting an experiment, administrators can note which version they are running and which feature they are using. It also enables the sharing of custom extensions with other researchers in the community. All experiment settings can be exported and shared as configuration files alongside study results. These settings files can include precise specifications for the user interface (e.g., item placement and size, the list of feedback

options, and the color schema). By doing so, Informfully enables controlling for item presentation across studies to recreate the exact same look and feel. Furthermore, our platform provides affordances for measuring user interactions, i.e., collecting raw behavioral data. Item clicks, views, and progression are all done with a clearly defined set of functions and level of granularity. This ensures that all experiments measure panelists' responses in exactly the same way.

4 Technical Documentation

Informfully is designed in a modularized way; it has an independent front end, back end, and a dedicated interface for communicating with external services. The separation of these core components enables the platform to be adapted to various use cases and to accommodate specific experiment needs. Section 4.1 first provides a high-level overview of the Informfully architecture. Next, Section 4.2 focuses on the app and website front end, and Section 4.3 on the server back end. Section 4.4 exemplifies communication with external services by showcasing data exchange with content scrapers, and Section 4.5 examines how external recommendations can be forwarded to Informfully. Finally, Section 4.6 describes the deployment process for running an instance of Informfully Research Platform.

4.1 Application Architecture

Informfully (front and back end) is built with React Native,¹⁰ Meteor,¹¹ and MongoDB.¹² We chose React Native because it allows us to reuse components across both the website and compatible mobile operating systems, Android and iOS, for cross-platform deployment. Given its widespread use, numerous resources are available online to help researchers become familiar with the underlying framework. Our decision to use Meteor for the system interfaces' backend leverages its support for JavaScript and integration with React Native. Regarding the database, we use MongoDB collections that store JSON-like documents. They integrate seamlessly with the Meteor framework and are the default option supported by this framework. During the design and development phase, we deliberately tried to minimize dependencies. Communication between the front end, back end, and external services is implemented via a shared interface in a blackboard architecture style [11]. Figure 4 shows an overview of the main components of our solution and how they communicate. We now outline the details.

App and Website Front End. Informfully's front end elements (left part of Figure 4) are the only components visible to end users. Experiment participants exclusively see the *app interface* when participating in a user study (see "App" in Figure 4), while the experiments are managed via the *administration website* (setting up experiments, user surveys, and creating user accounts, see "Website" in Figure 4). The administration website provides an overview of all experiments. The app interface, in contrast, only displays user-specific content (e.g., item feed).

Server Back End. Informfully's back end Meteor server (middle part of Figure 4) provides three main services: 1) *storage services and interfaces* for external RS, 2) *website functionalities* (user management, survey tool, and experiment configuration), and 3) *content scrapers* written in Python. Communication between components of different services occurs via database updates, adhering to the aforementioned blackboard architecture. This allows each component of Informfully to operate independently and to be easily exchanged if desired.

External Services. Informfully interacts with external resources (right part of Figure 4) such as content scrapers for items and recommender systems for personalized item feeds. As Informfully is algorithm-agnostic,

¹⁰Official website of React Native: <https://reactnative.dev>

¹¹Official website of Meteor: <https://meteor.com>

¹²Official website of MongoDB: <https://mongodb.com>

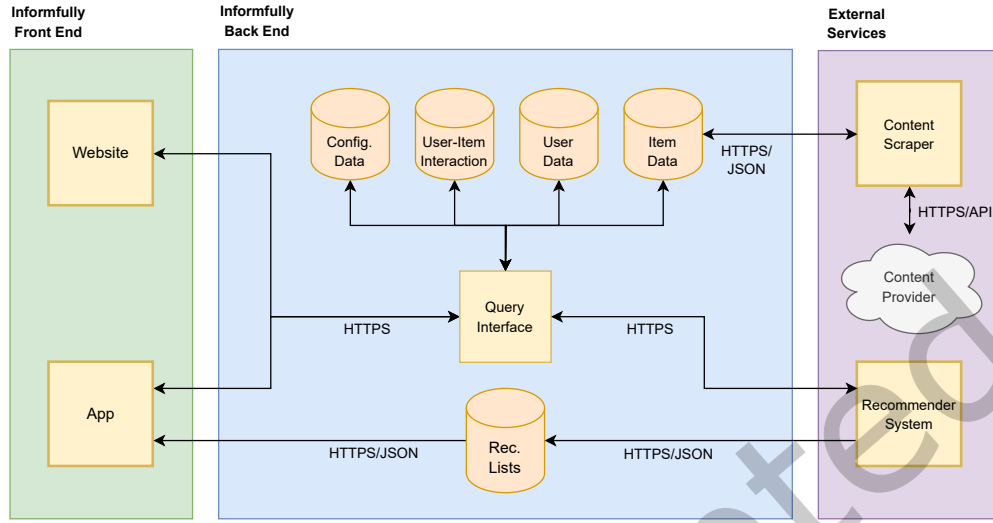


Fig. 4. Application architecture of the Informfully Research Platform. The front end (left in green) includes the app and website. The server back end (middle in blue) hosts the recommendation lists, the survey tool, user management, and the content scrapers. External services are shown on the right in purple.

sending/writing recommendation lists happens via JSON files in a recommendation exchange format (cf. Section 4.5 Listing 1 for details on these JSON files).¹³

Communication. Communication between the Informfully front end and back end occurs via HTTPS and is handled by the Meteor library. Communication between external services and the back end is managed by reading and writing to shared collections in the database. This creates an open and modular architecture that enables Informfully to accommodate recommendations from third-party RS frameworks (e.g., [2, 13, 43, 47]) without requiring any system integration.

Item Scraping. To gather content (e.g., from external sources such as websites), we offer a collection of Python scripts for scraping and processing item entries. They can be extended with source-specific API calls.¹⁴ Scraping of items and updating recommendations can be managed using a cron job to run independently of any recommendation routine.

¹³Stand-alone package of Informfully recommender algorithms on GitHub: <https://github.com/Informfully/Recommenders>

¹⁴Stand-alone package of Informfully scrapers on GitHub: <https://github.com/Informfully/Scrapers>

Component Interaction. With both the front end and the back end powered by Meteor, changes are reflected automatically once the relevant data collections have been updated. Informfully runs asynchronously and independently of any external dependency; no synchronization is needed for internal and external components. The performance delivered to the end user, e.g., the participant in the experiment, is not affected by potentially slow scraper or recommender implementations, which in other architectures might require the system to halt while they run and update.

4.2 Front End Overview

This section outlines the customizable elements of the front end. The focus is on the functionality of these components and how they can be changed to fit specific experimental needs. When designing the front end, the goal was to create an interface similar to a commercial RS application, such as a news or streaming app, in terms of both interaction and appearance. We used the usability and accessibility features identified in our initial literature review (please see Table 2 for details) as a guideline for our final design decisions. In doing so, our goal is to increase external validity in user studies, ease of use, and overall user satisfaction.

Localization. The app front supports multiple languages. Informfully is currently localized in English, French, and German. Localization and management of displayed text for buttons are handled using the i18next library.¹⁵ Each language has a dedicated language file that stores all translations.¹⁶ Creating a new localization involves copying an existing language file and translating its entries. This gives experimenters the freedom to add new localizations and to change button labels to account for the specific needs of the experiment being conducted.

Color Mode. Informfully comes with two color schemes: light and dark mode. The color mode matches the device’s system-wide settings. However, users can manually change the color mode in the app’s settings menu. The color constants are set globally via a singleton that triggers updates to React components when the values change.¹⁷ The centralized color definition enables researchers to easily customize and rebrand the app to meet their specific needs.

Offline Functionality. To keep the data on the device in sync with the server, we rely on the Meteor library’s functionality when the client is connected to the internet. When used offline, the app records and caches all interactions, which are automatically uploaded to the server when it is online again. We regard the client as the single source of truth; any interaction data on the server must ultimately reflect the locally cached interactions. We added a custom queuing mechanism on top of the Meteor functionality; once the device reconnects to the internet, local data is always prioritized and overwrites server data.¹⁸ This custom scheduler is an optional feature that can be disabled to allow for server-side changes to be applied to the local storage.

Item Container. Informfully contains a general-purpose item container for displaying images and text (see Figure 1B). The image can be replaced with a built-in media player for “serving” recommended video and audio content. The files must be provided as MP3s (for audio) or MP4s (for video). For audio and video files, a list of timestamps and actions is provided. These actions include playing, stopping, pausing, entering full-screen mode, fast-forwarding, and rewinding. This content-agnostic approach, utilizing a general-purpose item container, enables Informfully to support a broad range of use cases.

¹⁵Official website of i18next: <https://i18next.com>

¹⁶See code repository: <https://github.com/Informfully/Platform/tree/main/frontend/lib/i18n/locales>

¹⁷See code repository: <https://github.com/Informfully/Platform/blob/main/frontend/styles/DynamicColors.js>

¹⁸See code repository: <https://github.com/Informfully/Platform/tree/main/frontend/lib/meteorOffline>

Reactive UI. Informfully's front end is written in JavaScript and uses Node.js as a development environment.¹⁹ In addition to using the buttons in the app, users can also swipe left to go back in the item browsing view. While swiping provides the fastest, most comfortable method seen in other apps, we kept the buttons to blend with existing apps and increase accessibility. All interactions with UI elements (selection, swipe, and scroll) trigger a state change in the app and are automatically recorded in the back end.²⁰ While the device is connected to the internet, any operation that requests data from the server features a loading animation, e.g., for loading and rating articles.

4.3 Back End Overview

The modular design of Informfully enables users to swap out specific components, making it future-proof and easily extensible. The key design aspect that enables the architecture shown in Figure 4 is the communication between components through data collection. Here, we use the schema-free solution provided by MongoDB.²¹ This allows greater flexibility, for example, in adding new item properties to process all data related to the platform. Alternatively, a relational database could have been used, offering potential benefits in terms of speed and scalability. Ultimately, a schema-free approach was chosen, as multiple parallel experiments on the same server may record and utilize slightly different document definitions and datapoints in their user experiments.

Not enforcing a global schema allows our solution to be flexible enough to host all experiments in the same database without requiring migrations or redefining the underlying data schema. With the current version of Informfully, changing the source code is sufficient, as the used database schema is defined implicitly there. To illustrate this capability, this section presents a specific instance of how the tool was used as a news aggregator for illustration purposes (for more details on the news app instance, please see [21–23]). It introduces the back end as well as how to customize and extend the existing platform by providing an overview of its two main workflows: 1) processing items and user interactions, and 2) managing experiments and user groups.²²

Item Processing. The back end item view, as shown in Figure 5, provides an overview of item-related collections and their management during the recommendation process. This includes 1) the users and all collections that automatically record their interactions (orange headings in the Figure), as well as 2) the items and all collections managed by external services (green headings). The orange collections are managed internally, and their content may change within a recommendation cycle. The green collections are static and remain unaffected by user interactions.

A recommendation cycle in Informfully has two phases. First, external services provide items²³ recommendations, and optional explanations. Second, the new content is shown to users, and their implicit/explicit feedback is automatically recorded. These data can then be used in the subsequent recommendation cycle.²⁴ When extending/customizing the back end, experimenters can add and record additional experiment-specific attributes to the interaction collections.

¹⁹For a complete overview of the stack and libraries used in this project, please see the online documentation: <https://informfully.ch/docs/install.html>

²⁰This recording allows for a complete reconstruction of the user session. Each interaction record lists the unique name of the UI element (e.g., the UI mask for the detailed item view), the action performed (e.g., a left swipe), and a timestamp. Please see the online documentation for more details: <https://informfully.ch/docs/database.html#pageviews>

²¹Schema-free solutions enable adapting data collections without requiring schema migration.

²²Please note that the overview provided in this section provides a simplified view of the data landscape. The full list of all available collections and documentation of all their attributes is available in the project documentation: <https://informfully.ch/docs/database.html>

²³Media files (i.e., images, audio, and video) are referenced in the collection via their URLs. This allows greater flexibility, as both local and external files can be used without changing the codebase.

²⁴All collections in Figure 5 are accessible to external services, but the orange-headed components are read-only.

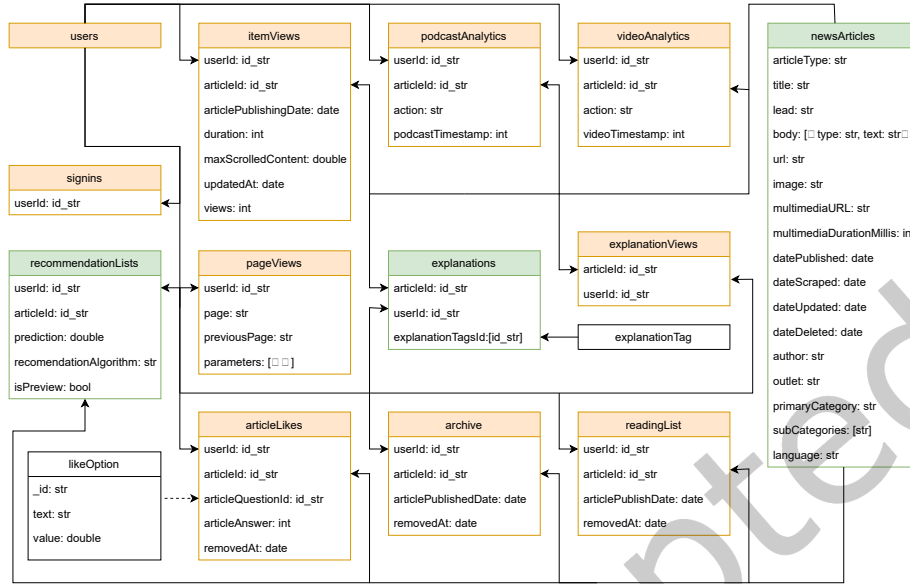


Fig. 5. Diagram of the data structure in the Informfully database related to articles and user interactions. Each table represents a MongoDB collection, except for tables with white headers, which denote inline structures stored in another collection. Dotted arrows hint at an inline data structure while solid arrows denote ID references. For all collections, the createdAt timestamp and the unique ID field `_id` were omitted.

Experiment Management. The experimental view of the back end in Figure 6 presents an outline of how experiments are stored by default. This includes 1) collections managed by experimenters via the administration website (shown with blue headings in the Figure), as well as 2) user collection and their interactions on an experiment level (orange heading).

The rationale for these three collection types is that the back end logic separates the data.²⁵ Collections created via the administration website can only be used within the scope of the experiment to which they are assigned. I.e., surveys and user groups are tied to an experiment and cannot be shared or reused. Users can only be assigned to one user group and, therefore, to only one experiment at a time. However, the user group assignment can be changed (e.g., to enroll participants in follow-up experiments).

4.4 Scraper Pipeline

Informfully includes a collection of dedicated content scrapers²⁶ written in Python with MongoDB for persistent storage of items. Each scraper module (called `scrape.py` in Figure 7) is required to implement a scraping function `scrape()`. Splitting the scraping task into independent modules allows for source-specific scrapers. The main program `main.py` will import all scraper modules and call their `scrape()` function, expecting them to return a list of items.

²⁵Content managed by external services, i.e., items and recommendation lists, is not included. It is up to the administrators to decide whether these collections can be used across experiments.

²⁶Thanks to the modular architecture of Informfully, experimenters can provide their own scrapers as long as they adhere to the standard for saving articles in the database.

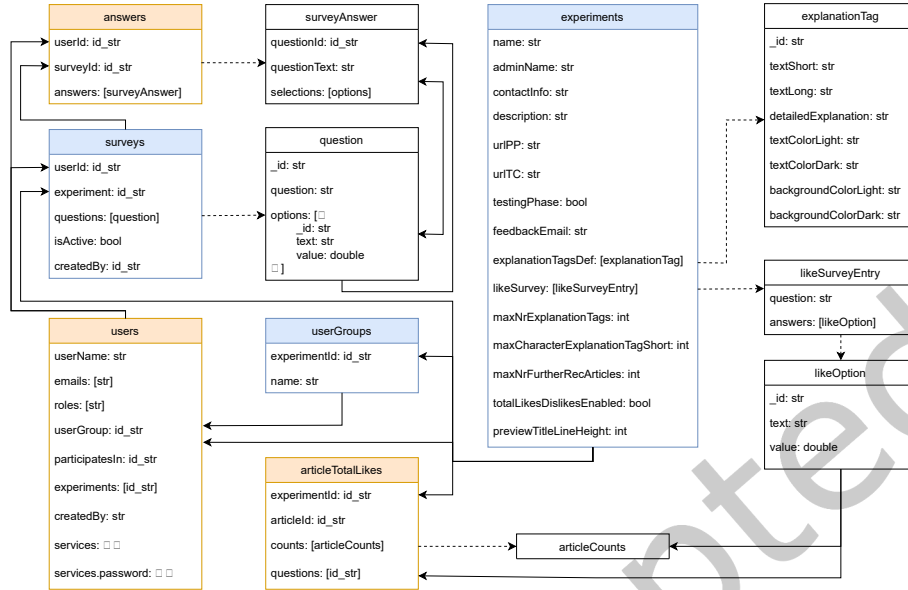


Fig. 6. Diagram of the data structure in the Informfully database related to user and experiment setup. Each table represents a MongoDB collection, except for tables with white headers, which denote inline structures stored in another collection. Dotted arrows hint at an inline data structure while solid arrows denote ID references. For all collections, the createdAt timestamp and the unique ID field `_id` were omitted.

Functionalities shared among the scrapers are stored in a separate utility file (`utils.py`), such as a function to create an item object for MongoDB. Before storing the items²⁷ to the document collections, the scraper performs a series of post-processing steps. The current post-processing pipeline for articles includes the `normalizer.py` script, which applies the same punctuation, date, and currency formats to all scraped items and sources. A final script, `cleaner.py`, then removes and/or replaces any pre-defined words from items (e.g., outlet from news articles to white-label them). This pipeline can be extended to account for experiment-specific requirements.²⁸

4.5 Recommender Interface

To facilitate this communication between an RS framework and the back end, Informfully uses a JSON recommendation exchange file (JREX). Listing 1 provides a JREX example of a recommendation passed from the RS to the Informfully back end. JREX files can be sent to the back end database at any time, and recommendations are then automatically sent to users.

²⁷Default representation of a scraped item: <https://informfully.ch/docs/items.html>

²⁸For more extensive documentation of the scraper pipeline, please see the online documentation: <https://informfully.ch/docs/scrapers.html>

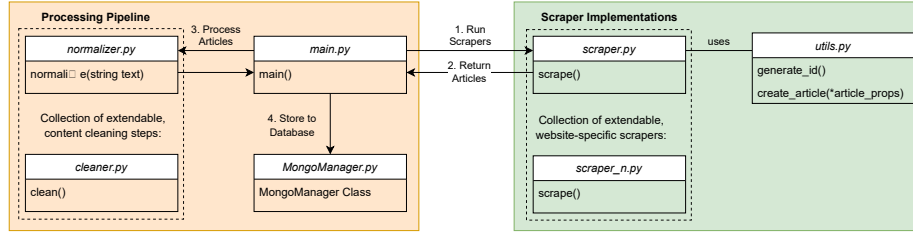


Fig. 7. Class diagram of the modularized scraper implementation and processing pipeline for scraping items from online resources. The orange part on the left controls the main tasks of scraping, cleaning, and storing items. The green part on the right details the scraping modules.

Listing 1. JREX format for updating personalized item recommendations.

```

1 {"recommendationLists": [
2   { "ID": ObjectId("dbdwHPsadszYvgP"),
3     "userID": "ksgsouZYPvBAGiQb",
4     "itemID": "632013714366",
5     "prediction": 0.117,
6     "recommendationAlgorithm": "NMF",
7     "isPreview": True,
8     "createdAt": 2024-05-21T12:19:40.229
9   }
]
```

A valid recommendation must feature a `userID` (target user ID), `articleId` (target article ID), and `prediction` (relevance score). Optional attributes include `recommendationAlgorithm` (algorithm identification for explainability purposes), `isPreview` (defining the visualization mode), and `createdAt` (timestamp). The attributes used in the JREX file are as follows:

ID (ObjectID): Unique Object ID used for indexing.

userID (String): ID of target user.

itemID (String): ID of target item.

prediction (Double): Prediction score that determines the position of the item within the recommendation list. The higher the score, the further up the item is placed in the news feed. Precision, upper, and lower limits of the score can be customized.

recommendationAlgorithm (String): Algorithm used to calculate the recommendation. Can optionally include an explanation of why this item was recommended.

isPreview (Boolean): The front end can display (or feature) items in a preview mode (with the item text and image across the entire screen, see top article in Figure 1A). Alternatively, items can be shown using a square thumbnail and title-only (see the bottom four articles in Figure 1A). If set to `TRUE`, the large preview is used, and the compact view is set to `FALSE`.

createdAt (Date): Timestamp that records when the item recommendation was created.

The idea behind exchanging recommendations via JREX files is to integrate Informfully as seamlessly as possible into the existing RS infrastructure. We provide a reference implementation of creating JREX files with

sample users and items²⁹ together with a tutorial on accessing/interfacing with the back end database.³⁰ By default—or in the absence of any content curation by a recommender algorithm—users are shown all items in the document collection of the item archive (cf. Figure 4), sorted by item creation date from new to old.

4.6 Project Setup

We conclude the technical section with an overview of Informfully’s deployment. In doing so, we focus on the two main steps of 1) setting up a local development environment for testing and 2) deployment of the server component.³¹ To deploy the application, both the development environment for building the mobile apps, including concrete server addresses, and the system deployment tasks must be completed.

Development Environment. After downloading the codebase from the Informfully Platform GitHub repository, we recommend installing a package manager to handle all the relevant dependencies. The current version of Informfully utilizes Yarn (v1.22 or higher) and npm (v8.1 or higher) as dependency managers, along with Expo CLI, a command-line tool that streamlines tasks such as project setup, building, and deployment. The list of required library dependencies is in the *package.json* files, which are in both the *frontend* and *backend* subfolders of the project. The *frontend* folder contains all the files required to build the app. The *backend* folder contains all server components, including the website. For app development, users must install either Xcode for iOS development or an Android development environment, such as Android Studio.

System Deployment. Local development and public release of the platform require running a server to host the back end. The app connects to this server via a URL specified in the *SERVER* constant in the *App.js* file.³² We provide a script that automatically configures and deploys the website and the back end on the server. This automatic approach bundles commands that perform a series of operations, from creating a *Node.js* app bundle, transferring the data, and installing all relevant dependencies. These commands are bundled in a *build.sh* bash script.³³ Before using the script, the *snippets.sh* file must be configured with the correct settings for the new server and target directories. Both the *build.sh* and *snippets.sh* files are located in the root directory of the back end. We recommend using *Phusion Passenger*³⁴ for a secure deployment that handles process management automatically, while offering better security and performance capabilities, as compared to the traditional node-based servers. The external scraper and recommender components required no special deployment, as long as they wrote to the predefined database collections.³⁵

4.7 Data Management

An external RS can connect to the Informfully back end via the query interface (see Figure 4) to access user data, item data, and user-item interactions. Table 3 provides an overview of a few selected key collections that can be accessed.³⁶ There are four categories of data collections: 1) user data (e.g., *pageViews*, *users*, and *signins*), 2) item data (*items*), 3) user-item interactions data (e.g., *archive*, *favorites*, and *itemViews*), and 4) configuration

²⁹ The documentation and a reference implementation on how to create JREX files are available online: <https://informfully.ch/docs/recommendations.html>

³⁰ Documentation on accessing the back end database: <https://informfully.ch/docs/compass.html>

³¹ For the detailed step-by-step tutorial, we refer to the complementary “Getting Started” guide provided in the online documentation: <https://informfully.ch/docs/install.html>

³² See code repository: <https://github.com/Informfully/Platform/tree/main/frontend>

³³ See code repository: <https://github.com/Informfully/Platform/tree/main/backend>

³⁴ Official website of Phusion Passenger: <https://phusionpassenger.com>

³⁵ We provide a sample RS back end for calculating user recommendations that adopts JREX. Online documentation for recommender: <https://informfully.ch/docs/recommenders.html>

³⁶ Please note that this is only a limited selection. The full list of all available collections and a documentation of all their attributes is available on the project wiki: <https://informfully.ch/docs/database.html>

collections for setting up and running experiments (e.g., experiments, surveys, and userGroups). All collections listed in Table 3 are schema-free. We recommend using the built-in MongoDB option to directly export data from the database in JSON and CSV formats.³⁷

Being able to export the data to these open formats makes publishing the data and using it for other researchers easier than, for example, with binary SQL database dumps, which can be cumbersome to resolve compatibility issues. Furthermore, with direct access to the database, users of our platform are free to build their own backup/versioning systems and export data in such a format that it fits into their existing data analysis pipelines.³⁸

Table 3. Excerpt of available database collections of the back end that can be accessed and exported during and after the experiment for subsequent data analysis.

Collection	Description
answers	User answers to in-app surveys.
archive	List of bookmarked items of all users.
items	Collection of items, incl. properties.
itemLikes	User likes/dislikes for all items.
itemViews	Items accessed by users.
pageViews	User navigation history through the app.
readingList	List of favorite items of all users.
signins	User sign-in history.
users	Collection of user properties.

With Informfully, our focus is to record all possible user interactions within the app and provide them as raw data for download (i.e., CSV or JSON files). In this context, however, it is important to note that our platform does not provide any scripts or other functionality for downstream data analysis and evaluation tasks. Researchers are encouraged to write their own scripts to produce data in the formats used by their existing data pipelines and analysis workflows.

5 Case Studies

The goal of this section is to demonstrate how Informfully can be used to support empirical research, how it can be customized, and that its feature set has been field-tested. The scope is limited to the news recommendations use case. In this domain, we used Informfully to conduct two studies in Switzerland (study CH’20 in 2020, see [22] for details, and study CH’22 in 2022/2023) that focused on recommending news articles (text and images), podcasts, and videos to cover all item modalities. The tool was used in Germany (study DE’22 in 2022, see [21]), which allowed us to test decentralized deployment (user base, recommenders, and scrapers in Germany communicating with the back end in Switzerland). One final study in the Netherlands (study NL’23 in 2023, see [23, 34]) made extensive use of the survey tool for creating quizzes. In addition, two offline benchmarking studies tested the reliability and scalability of the [15, 19]. Each deployment enabled us to gather valuable feedback, enabling iterative improvements to the platform.³⁹

Table 4 summarizes the key metrics of the user studies and the features they used.⁴⁰ Metrics include the number of participants, user groups, recommender algorithms, surveys, and scrapers. We also list the study durations in

³⁷We open-sourced a dataset from a previous experiment [23, 34] to serve as an example of the type and quantity of data that is automatically being recorded: <https://github.com/Informfully/Datasets>

³⁸For sharing the back end configurations and other server scripts, we recommend sharing an updated Docker Container: <https://informfully.ch/docs/docker.html>

³⁹We limit the selection of experiments to user studies. For offline evaluation of design aspects, please see [41, 42].

⁴⁰We limit the overview to a summary of the data and refer to the relevant publications for a detailed discussion.

days, the number of unique items scraped, the recommendation intervals, and the sizes of the item lists. Finally, it lists all user-item interactions and the featured item modalities: text (t), image (I), audio (a), and video (v).

Table 4. Overview of key metrics of past user studies conducted with Informfully, showcasing the customizability of the tool to accommodate experiment-specific needs.

Metric	CH'20	CH'22	DE'22	NL'23
Participants	151	941	143	283
User groups	3	5	2	12
Number of Algorithms	3	5	2	4
Surveys	3	4	0	12
Scrapers	6	12	1	6
Duration (days)	35	183	28	14
Unique items	7,287	41,340	2,191	442
Recommendation interval	20 min.	10 min.	24 hrs.	24 hrs.
Recommendation list size	50-300	50-150	15-50	26
User interactions	55.3k	1.21m	17.9k	40.1k
Modalities	t, i	t, i, a, v	t, i	t, i

To allow for a better assessment of the capabilities of Informfully for reproducible research, we would like to discuss these in greater detail. The goal in studies CH'20 and CH'22 was to look at changes in news consumption behavior and satisfaction with news media. Users were randomly assigned to a one-sided, a diversity-optimized, or a chronological news recommender algorithm. As a news aggregator, Informfully collected items from various news outlets and served custom news recommendations tailored to different political viewpoints. During the experiment, the app tracked reading progress and prompted surveys on political topics and overall user satisfaction with news media. These studies presented the first real-world experiments conducted with the Informfully platform. Asking participants for feedback at the end of the experiments was invaluable and led to numerous improvements (e.g., swipe controls, new menu controls, dark/light mode, in-app tutorials, and multilingual on-screen text elements).

Study DE'22 focused on changes in voting behavior and knowledgeability (in terms of political actors and party agenda). The app provided users with daily updates on the ongoing election campaign at both the federal and local levels. A particular focus was given to minority parties, who received greater exposure in the app through an increased item count compared to majority parties and by being placed higher up in the recommendation list, with a larger picture and a visible article lead. This experiment enabled us to enhance our custom caching module for tracking user interactions and reduce overall latency when serving recommendations.

Study NL'23 looked at how 1) position nudges (varying on-screen item placement) and 2) accessibility nudges (varying text complexity) impact item consumption and knowledgeability. Changes in consumption rate were measured using the automatic interaction logging of Informfully. Effects on knowledgeability were assessed through regular in-app surveys that asked participants multiple-choice questions about the items they had consumed. It is the first instance of creating a user pool and re-enlisting participants for a subsequent study. Participants were automatically transferred to a new experiment for the second week. This required reworking the logic behind group membership, allowing it to be changed and carried over to new user experiments.

Finally, we would like to emphasize the flexibility Informfully offers, allowing experiments to adjust their scope and duration. Study CH'22 and NL'23 both required additional recruitment waves, necessitating the creation of new user accounts and extending the experiment timeline. Thanks to the high degree of automation, these

changes could be accommodated effortlessly by creating new user accounts in the web interface and adding them to existing groups.

6 Limitations and Future Work

The Informfully Research Platform is currently under development. As such, several limitations apply. Most prominently, there has not yet been a systematic scientific evaluation of the platform’s accessibility features and interfaces. The design was guided by feedback and change requests from the open source community and from research partners who had previously used the system. Similarly, scalability benchmarking was conducted only for the expected workload and the specific configuration of the user studies features in Table 4.

While the tool includes the capabilities to manage and maintain users, it does not give researchers access to panelists. Unlike platforms such as *POPPOX* [8, 27] that maintain their own active participant pool, Informfully requires external recruitment. The app can only be used for active online experiments, and there are currently no default use cases beyond them.

Looking at reproducibility in general, it is important to note that any experiment involving people will produce a variance in the output. Despite using the same setup and parameters, several key factors, including the setting, the cultural background of the panelists, and the selection of items, can significantly impact the results and outcome of the study. Therefore, we encourage researchers to also share details beyond the technical configuration of the experiment (e.g., demographic information) and to verify whether (re-)deploying a specific setup is reasonable and feasible.

In terms of future work, several upcoming features are planned. To ensure parity across platforms, Informfully is currently using the mobile user interface. Installing the web version on desktops will not alter the app’s layout. Future implementations, however, should support different interface styles (e.g., arranging items not only in a list but also in a grid). Furthermore, the presentation currently allows for only one multimedia item per article (i.e., one image, audio, or video file). We are planning to extend the platform to accommodate a larger number by adding a carousel to browse different images in the header of an item and by allowing multimedia elements to be placed anywhere within the body.

Looking at item customization, we are currently working on features to allow us to display personalized images based on user preferences [16, 17, 24]. This extension complements the app’s customization, allowing researchers to tailor it to their specific needs. Customization is currently limited to arranging items in the home feed and enabling/disabling item ratings and related items in the article view. While JREX allows passing on algorithm parameters to switch between different interface designs (see Listing 1), Informfully includes only one set of default interfaces for item presentation. Additional sets would need to be created first.

Furthermore, participants and researchers suggested different social features for our experiments, which we intend to implement. This includes comment sections, user chats, social media integration, and the ability to share content with friends, which may involve active content moderation and data sharing agreements with third-party services. Extended customization for item ratings is also planned. This includes star and Likert scale ratings, as well as the like/dislike option.

7 Conclusion

Informfully presents an ongoing endeavor to create a domain-independent platform for content distribution that facilitates all the tools necessary for conducting empirical research. Such empirical user studies are crucial for assessing the performance of RSs. The currently predominant way of evaluating algorithmic curation systems in an offline setting will provide only limited insight compared to empirical studies with real-world user interactions. Our platform addresses this shortcoming by providing a tool that makes conducting user studies more accessible. It seamlessly integrates functions for creating user accounts, organizing experiments, assigning experimental

groups, managing surveys, monitoring experiment participants, and providing automated tracking capabilities for both short-term and longitudinal user studies. It provides an interface for multimedia item recommendations in a list view and real-time recording of user interactions. Informfully has a proven track record for text, image, audio, and video recommendations. To the best of our knowledge, there is currently no other open source tool that offers a more extensive suite of features for field experiments. By open-sourcing our Informfully platform, we hope it will become a stepping stone towards more user-focused research and spark interest in conducting mode user studies to evaluate RSs.

Acknowledgments

This work was partially funded by the Digital Society Initiative (DSI) of the University of Zurich, through grants from the DSI Excellence Program, the Hasler Foundation, and the Swiss Federal Office of Communications (OFCOM). We would like to thank our students Markus Göckeritz, Christopher Schwizer, Luca Weibel, Valérie Nyffeler, Lukas Yu, Emanuel Graf, Vladimir Donkov, Hasan Atak, Hongjie Guan, and Wenzhe Li for their contributions to developing the Informfully Research Platform.

References

- [1] Gediminas Adomavicius, Jesse Bockstedt, Shawn Curley, and Jingjing Zhang. 2019. Reducing recommender systems biases: An investigation of rating display designs. *MIS Quarterly* 43, 4 (2019), 19–18.
- [2] Vito Walter Anelli, Alejandro Bellogin, Antonio Ferrara, Daniele Malitesta, Felice Antonio Merra, Claudio Pomo, Francesco Maria Donini, and Tommaso Di Noia. 2021. Elliot: A comprehensive and rigorous framework for reproducible recommender systems evaluation. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 2405–2414.
- [3] Maxim Bakaev and Tatiana Avdeenko. 2012. User interface design guidelines arrangement in a recommender system with frame ontology. In *Database Systems for Advanced Applications: 17th International Conference, DASFAA 2012, International Workshops: FlashDB, ITEMS, SNSM, SIM 3, DQDI, Busan, South Korea, April 15-19, 2012. Proceedings* 17. Springer, 311–322.
- [4] Joeran Beel and Haley Dixon. 2021. The ‘unreasonable’ effectiveness of graphical user interfaces for recommender systems. In *Adjunct Proceedings of the 29th ACM Conference on User Modeling, Adaptation and Personalization*. 22–28.
- [5] Joeran Beel, Bela Gipp, Stefan Langer, and Corinna Breiting. 2016. Paper recommender systems: a literature survey. *International Journal on Digital Libraries* 17 (2016), 305–338.
- [6] Joeran Beel and Stefan Langer. 2015. A comparison of offline evaluations, online evaluations, and user studies in the context of research-paper recommender systems. In *Research and Advanced Technology for Digital Libraries: 19th International Conference on Theory and Practice of Digital Libraries, TPDL 2015, Poznań, Poland, September 14-18, 2015, Proceedings* 19. Springer, 153–168.
- [7] Abraham Bernstein, Claes De Vreese, Natali Helberger, Wolfgang Schulz, Katharina Zweig, Lucien Heitz, Suzanne Tolmeijer, et al. 2021. Diversity in News Recommendation. *Dagstuhl Manifestos* 9, 1 (2021), 43–61.
- [8] Robin Burke and Michael Ekstrand. 2025. Conducting Recommender Systems User Studies Using POPROX. In *Adjunct Proceedings of the 33rd ACM Conference on User Modeling, Adaptation and Personalization*. 1–2.
- [9] Dan Cosley, Shyong K Lam, Istvan Albert, Joseph A Konstan, and John Riedl. 2003. Is seeing believing? How recommender system interfaces affect users’ opinions. In *Proceedings of the SIGCHI conference on Human factors in computing systems*. 585–592.
- [10] Patrik Dokoupil and Ladislav Peska. 2023. Easystudy: Framework for easy deployment of user studies on recommender systems. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1196–1199.
- [11] Lee D Erman, Frederick Hayes-Roth, Victor R Lesser, and D Raj Reddy. 1980. The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty. *ACM Computing Surveys (CSUR)* 12, 2 (1980), 213–253.
- [12] Florent Garcin and Boi Faltings. 2013. Pen recsys: A personalized news recommender systems framework. In *Proceedings of the 2013 International News Recommender Systems Workshop and Challenge*. 3–9.
- [13] Guibing Guo, Jie Zhang, Zhu Sun, and Neil Yorke-Smith. 2015. Librec: A java library for recommender systems. In *CEUR Workshop Proceedings*, Vol. 1388. RWTH Aachen University.
- [14] Jaron Harambam, Dimitrios Bountouridis, Mykola Makhortykh, and Joris van Hoboken. 2019. Designing for the Better by Taking Users into Account: A Qualitative Evaluation of User Control Mechanisms in (News) Recommender Systems. In *Proceedings of the 13th ACM Conference on Recommender Systems (Copenhagen, Denmark) (RecSys ’19)*. Association for Computing Machinery, New York, NY, USA, 69–77. doi:10.1145/3298689.3347014
- [15] Lucien Heitz. 2023. Classification of Normative Recommender Systems. In *Proceedings of the First Workshop on the Normative Design and Evaluation of Recommender Systems*.

- [16] Lucien Heitz, Abraham Bernstein, and Luca Rossetto. 2024. An Empirical Exploration of Perceived Similarity between News Article Texts and Images. In *Working Notes Proceedings of the MediaEval 2023 Workshop*.
- [17] Lucien Heitz, Yui Kwan Chan, Hongji Li, Kerui Zeng, Abraham Bernstein, and Luca Rossetto. 2024. Prompt-based Alignment of Headlines and Images Using OpenCLIP. In *Working Notes Proceedings of the MediaEval 2023 Workshop*.
- [18] Lucien Heitz, Julian A Croci, Madhav Sachdeva, and Abraham Bernstein. 2024. Informfully - Research Platform for Reproducible User Studies. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 660–669.
- [19] Lucien Heitz, Oana Inel, and Sanne Vrijenhoek. 2024. Recommendations for the Recommenders: Reflections on Prioritizing Diversity in the RecSys Challenge. In *Proceedings of the Recommender Systems Challenge 2024*.
- [20] Lucien Heitz, Runze Li, Oana Inel, and Abraham Bernstein. 2025. Informfully Recommenders – Reproducibility Framework for Diversity-aware Intra-session Recommendations. In *Proceedings of the 19th ACM Conference on Recommender Systems*. ACM New York, NY, USA, 792–801. <https://doi.org/10.1145/3705328.3748148>
- [21] Lucien Heitz, Juliane A Lischka, Rana Abdullah, Laura Laugwitz, Hendrik Meyer, and Abraham Bernstein. 2023. Deliberative Diversity for News Recommendations: Operationalization and Experimental User Study. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 813–819.
- [22] Lucien Heitz, Juliane A Lischka, Alena Birrer, Bibek Paudel, Suzanne Tolmeijer, Laura Laugwitz, and Abraham Bernstein. 2022. Benefits of diverse news recommendations for democracy: A user study. *Digital Journalism* 10, 10 (2022), 1710–1730.
- [23] Lucien Heitz, Nicolas Mattis, Oana Inel, and Wouter van Atteveldt. 2024. IDEA – Informfully Dataset with Enhanced Attributes. In *Proceedings of the Second Workshop on the Normative Design and Evaluation of Recommender Systems*.
- [24] Lucien Heitz, Luca Rossetto, Benjamin Kille, Andreas Lommatzsch, Mehdi Elahi, and Duc-Tien Dang-Nguyen. 2025. NewsImages in MediaEval 2025 – Comparing Image Retrieval and Generation for News Articles. In *Working Notes Proceedings of the MediaEval 2025 Workshop*.
- [25] Lucien Heitz, Krisztina Rozgonyi, Bojana Kostic, Deniz Wagner, and Julia Haas. 2021. *AI in Content Curation and Media Pluralism*. Organization for Security and Co-operation in Europe, Austria, Chapter 2, 56–70. <https://zora.uzh.ch/id/eprint/213723>
- [26] Natali Helberger. 2019. On the democratic role of news recommenders. *Digital Journalism* 7, 8 (2019), 993–1012.
- [27] Karl Higley, Robin Burke, Michael D Ekstrand, and Bart P Knijnenburg. 2025. What News Recommendation Research Did (But Mostly Didn't) Teach Us About Building A News Recommender. *arXiv preprint arXiv:2509.12361* (2025).
- [28] Rashedul Islam, Rofiqul Islam, and Tohidul Mazumder. 2010. Mobile application and its global impact. *International Journal of Engineering & Technology* 10, 6 (2010), 72–78.
- [29] Dietmar Jannach and Christine Bauer. 2020. Escaping the McNamara fallacy: Towards more impactful recommender systems research. *Ai Magazine* 41, 4 (2020), 79–95.
- [30] Dietmar Jannach, Markus Zanker, Mouzhi Ge, and Marian Gröning. 2012. Recommender systems in computer science and information systems—a landscape of research. In *E-Commerce and Web Technologies: 13th International Conference, EC-Web 2012, Vienna, Austria, September 4-5, 2012. Proceedings 13*. Springer, 76–87.
- [31] R Leuener. 2017. NZZ Companion: How we successfully developed a personalised news application. Medium.
- [32] Runze Li, Lucien Heitz, Oana Inel, and Abraham Bernstein. 2025. D-RDW: Diversity-Driven Random Walks for News Recommender Systems. In *Proceedings of the 19th ACM Conference on Recommender Systems*. ACM New York, NY, USA, 558–563. <https://doi.org/10.1145/3705328.3748016>
- [33] Felicia Loecherbach and Damian Trilling. 2020. 3bij3—Developing a framework for researching recommender systems and their effects. *Computational Communication Research* 2, 1 (2020), 53–79.
- [34] Nicolas Mattis, Lucien Heitz, Philipp K Masur, Judith Moeller, and Wouter van Atteveldt. 2025. Nudges for News Recommenders: Prominent Article Positioning Increases Selection, Engagement, and Recall of Environmental News, but Reducing Complexity Does Not. *Journal of Communication* (2025), jqaf019. <https://doi.org/10.1093/joc/jqaf019>
- [35] Martijn Millecamp, Nyi Nyi Htun, Cristina Conati, and Katrien Verbert. 2020. What's in a user? Towards personalising transparency for music recommender interfaces. In *Proceedings of the 28th ACM Conference on User Modeling, Adaptation and Personalization*. 173–182.
- [36] Mats Mulder, Oana Inel, Jasper Oosterman, and Nava Tintarev. 2021. Operationalizing framing to support multiperspective recommendations of opinion pieces. In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*. 478–488.
- [37] Theodora Nanou, George Lekakos, and Konstantinos Fouskas. 2010. The effects of recommendations' presentation on persuasion and satisfaction in a movie recommender system. *Multimedia systems* 16 (2010), 219–230.
- [38] Theresa Neil. 2014. *Mobile design pattern gallery: UI patterns for smartphone apps*. " O'Reilly Media, Inc."
- [39] Georgios Paliouras, Alexandros Mouzakidis, Vassileios Moustakas, and Christos Skourlas. 2008. PNS: A personalized news aggregator on the web. *Intelligent interactive systems in knowledge-based environments* (2008), 175–197.
- [40] Lumpapun Punchoojit and Nuttanut Hongwarittorn. 2017. Usability studies on mobile user interface design patterns: a systematic literature review. *Advances in Human-Computer Interaction* 2017, 1 (2017), 6787504.
- [41] Luca Rossetto, Matthias Baumgartner, Narges Ashena, Florian Ruosch, Romana Pernisch, Lucien Heitz, and Abraham Bernstein. 2021. VideoGraph—towards using knowledge graphs for interactive video retrieval. In *International Conference on Multimedia Modeling*.

- Springer, 417–422.
- [42] Luca Rossetto, Matthias Baumgartner, Ralph Gasser, Lucien Heitz, Ruijie Wang, and Abraham Bernstein. 2021. Exploring Graph-querying approaches in LifeGraph. In *Proceedings of the 4th Annual on Lifelog Search Challenge*. Springer, 7–10.
 - [43] Aghiles Salah, Quoc-Tuan Truong, and Hady W Lauw. 2020. Cornac: A Comparative Framework for Multimodal Recommender Systems. *Journal of Machine Learning Research* 21, 95 (2020), 1–5.
 - [44] Holli Sargeant, Eliska Pirkova, Matthias C Kettemann, Marlena Wisniak, Martin Scheinin, Emmi Bevensee, Katie Pentney, Lorna Woods, Lucien Heitz, Krisztina Rozgonyi, Bojana Kostic, Deniz Wagner, Julia Haas, et al. 2022. *Spotlight on Artificial Intelligence and Freedom of Expression: A Policy Manual*. Organization for Security and Co-operation in Europe, Austria. <https://osce.org/representative-on-freedom-of-media/510332>
 - [45] Brendan Strahm, Colin M Gray, and Mihaela Vorvoreanu. 2018. Generating mobile application onboarding insights through minimalist instruction. In *Proceedings of the 2018 Designing Interactive Systems Conference*. 361–372.
 - [46] Kevin J Sullivan, William G Griswold, Yuanfang Cai, and Ben Hallen. 2001. The structure and value of modularity in software design. *ACM SIGSOFT Software Engineering Notes* 26, 5 (2001), 99–108.
 - [47] Zhu Sun, Di Yu, Hui Fang, Jie Yang, Xinghua Qu, Jie Zhang, and Cong Geng. 2020. Are we evaluating rigorously? benchmarking recommendation for reproducible evaluation and fair comparison. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 23–32.
 - [48] Panagiotis Symeonidis, Alexandros Nanopoulos, and Yannis Manolopoulos. 2009. MoviExplain: a recommender system with explanations. In *Proceedings of the third ACM conference on Recommender systems*. 317–320.
 - [49] Bogdan Walek and Vladimir Fojtik. 2020. A hybrid recommender system for recommending relevant movies using an expert system. *Expert Systems with Applications* 158 (2020), 113452.

Received 15 March 2025; revised 16 April 2026; accepted 22 May 2026